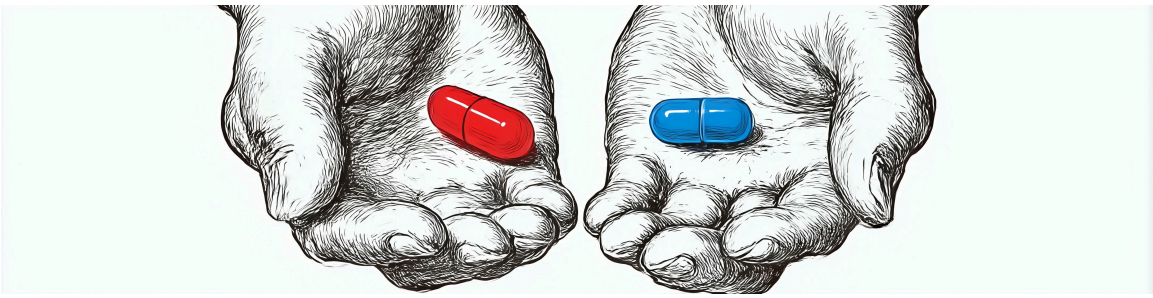
Verzweigung

Joern Ploennigs · AI4SC



> When you come to a fork in the road, take it.
>
> — Yogi Berra

Bedingtes Verzweigen

Ein Computer muss oft entscheiden wie Informationen verarbeitet werden sollen. Dafür muss er im Programm die Möglichkeit haben, bestimmte Statements oder Programmteile nur in bestimmten Fällen auszuführen. Hierfür bieten alle Programmiersprachen das Konzept der Verzweigung an.

Die einfachste Form der Verzweigung sind bedingte Verzweigungen, welche einen Code-Block nur bei Zutreffen einer logischen Bedingung ausführen. In Python ist dafür die `if`-Anweisung (Wenn-Dann) vorgesehen, welche einem Statement vorsteht, das nur bei Zutreffen einer Bedingung ausgeführt wird. Es ist üblich, die Statements auf eine neue Zeile zu schreiben. Dann *müssen* sie eingerückt werden.

```
bedingung=True
if bedingung:
    # wird nur ausgeführt wenn Bedingung True ist
    print(f"Der Block mit 'bedingung={bedingung}' wurde ausgeführt. 1")

bedingung=False
if bedingung:
    # wird nur ausgeführt wenn Bedingung True ist
    print(f"Der Block mit 'bedingung={bedingung}' wurde ausgeführt. 2")
```

```
Der Block mit 'bedingung=True' wurde ausgeführt. 1
```

> **Hinweis**
>
> In englischen Veröffentlichungen in der Informatik wird häufig auch im Schreiben das Wort `iff` verwendet, was für „if-and-only-if“ steht und explizit aussagt, dass eine strikte logische Aussage gemacht wird und kein natürlichsprachliches, weiches `if` gemeint ist.

Mit dem logischen Operator `not` kann man Bedingungen negieren und mit `and`, `or` und Klammern kombinieren.

```
bedingung=True
if not bedingung:
    print(f"Der Block mit 'bedingung={bedingung}' wurde ausgeführt.")

bedingung=0
if bedingung:
    print(f"Der Block mit 'bedingung'={bedingung} wurde ausgeführt.")

bedingung=2
if bedingung:
    print(f"Der Block mit 'bedingung'={bedingung} wurde ausgeführt.")

if not bedingung:
    print(f"Der Block mit 'bedingung={bedingung}' wurde ausgeführt.")
```

```
Der Block mit 'bedingung'=2 wurde ausgeführt.
```

Eine typische Frage im Code lautet: „Ist diese Liste leer?“ Dafür braucht man keinen Vergleich wie `len(meine_liste) == 0`. In Python kann man den Wahrheitswert direkt verwenden.

```
bauteile = []
if not bauteile:
    print("Die Liste ist leer.")

bauteile = ["Traeger", "Stuetze"]
if bauteile:
    print("Die Liste enthält Elemente.")
```

```
Die Liste ist leer.
Die Liste enthält Elemente.
```

Strings die leer und die Länge 0 haben entsprechen dem logischen Wert `False` und nicht leere dem logischen Wert `True`.

```
bedingung='' # leerer String
if bedingung:
    print(f"Der Block mit 'bedingung'={bedingung} wurde ausgeführt.")

bedingung=' '
if bedingung:
    print(f"Der Block mit 'bedingung'={bedingung} wurde ausgeführt.")
```

```
Der Block mit 'bedingung'=  wurde ausgeführt.
```

Das gleiche gilt für Listen und Dictionaries, wenn diese leer sind (`False`) oder befüllt (`True`).

```
bedingung=[] # leere Liste
if bedingung:
    print(f"Der Block mit 'bedingung'={bedingung} wurde ausgeführt.")

bedingung=['nicht_leer']
if bedingung:
    print(f"Der Block mit 'bedingung'={bedingung} wurde ausgeführt.")
```

```
Der Block mit 'bedingung'=['nicht_leer'] wurde ausgeführt.
```

Damit lässt sich im Code schnell prüfen ob z.B. bereits Elemente in einer Liste enthalten sind und diese z.B. weiterverarbeitet werden sollen.

Bei komplexeren Datentypen, wie Objekten, die den Wert `None` annehmen können, wird dieser auch als `False` interpretiert und alles andere als `True`, womit man wiederum prüfen kann, ob die Variable schon belegt worden ist.

Eine häufige Umformung kombinierter Bedingungen sind die De-Morgan-Regeln:

- `not (A and B)` ist gleichbedeutend zu `not A or not B`
- `not (A or B)` ist gleichbedeutend zu `not A and not B`

Solche Umformungen helfen, KI-generierten Code mit komplexen Bedingungen besser zu lesen oder in eine verständlichere Form zu bringen.

```
bedingung=None # nicht belegt
if bedingung:
    print(f"Der Block mit 'bedingung'={bedingung} wurde ausgeführt.")

bedingung="nicht_None"
if bedingung:
    print(f"Der Block mit 'bedingung'={bedingung} wurde ausgeführt.")

bedingung=None
```

```python
if bedingung is not None:
    print(f"Der Block mit 'bedingung'={bedingung} wurde ausgeführt.")

bedingung="nicht_None"
if bedingung is not None:
    print(f"Der Block mit 'bedingung'={bedingung} wurde ausgeführt.")
```

```
Der Block mit 'bedingung'=nicht_None wurde ausgeführt.
Der Block mit 'bedingung'=nicht_None wurde ausgeführt.
```

Wenn ... Dann ...

Die Grundidee einer Verzweigung lautet: Wenn eine Bedingung zutrifft, dann führe einen bestimmten Code-Block aus. Optional kann man zusätzlich festlegen, was im anderen Fall geschehen soll.

Die `if`-Anweisung

Die `if`-Anweisung startet in Python eine Verzweigung. Hinter `if` steht eine Bedingung, danach folgt ein Doppelpunkt. Der zugehörige Code-Block steht eingerückt in den folgenden Zeilen.

Beispiel: `if`-Anweisung

Im folgenden Beispiel wird die Ausgabe nur dann aktiviert, wenn `ausgabe_aktiv` den Wahrheitswert `True` hat. So kann man Berechnungen durchführen, aber die Ausgabe nur bei Bedarf anzeigen.

```python
ausgabe_aktiv = True
a = 10
b = 13
c = (a**2 + b**2) ** 0.5

if ausgabe_aktiv:
    print("Seitenlänge:" + str(c))
    umfang = a + b + c
    print("Umfang:" + str(umfang))
```

```
Seitenlänge:16.401219466856727
Umfang:39.40121946685673
```

Die Alternative

Oft kommt es vor, dass nicht nur etwas ausgeführt werden soll, wenn die Bedingung wahr ist. Der `else`-Block wird ausgeführt, wenn die Bedingung falsch ist. Dafür gibt es die `else`-Anweisung, welche ausschließlich nach einem `if` folgen darf und bei mehreren Zeilen auf gleicher Höhe eingerückt werden muss wie das `if`, das sie ergänzt.

```
bedingung=True
if bedingung:
    print(f"Der Block mit 'bedingung={bedingung}' wurde ausgeführt.")
else:
    print(f"Der alternative Block mit 'bedingung={bedingung}' wurde
ausgeführt.")
```

```
Der Block mit 'bedingung=True' wurde ausgeführt.
```

```
bedingung=False
if bedingung:
    print(f"Der Block mit 'bedingung={bedingung}' wurde ausgeführt.")
else:
    print(f"Der alternative Block mit 'bedingung={bedingung}' wurde
ausgeführt.")
```

```
Der alternative Block mit 'bedingung=False' wurde ausgeführt.
```

Ein typisches Beispiel ist das Abfangen von Divisionen durch 0. Bei Berechnungen kann man nicht immer garantieren, dass der Nenner ungleich 0 ist. Da eine Division durch 0 nicht definiert ist, führt sie zu einem Programmfehler und ggf. zu einem Programmabsturz.

```
zaehler = 5
nenner = 0
ergebnis = zaehler / nenner
```

```
ZeroDivisionError: division by zero
[31m---------------------------------------------------------------------------[
[39m
[31mZeroDivisionError[39m                            Traceback (most recent
call last)
[36mCell[39m[36m [39m[32mIn[10][39m[32m, line 3[39m
[32m      1[39m zaehler = [32m5[39m
[32m      2[39m nenner = [32m0[39m
[32m----> [39m[32m3[39m ergebnis = [43mzaehler[49m[43m
[49m[43m/[49m[43m [49m[43mnenner[49m

[31mZeroDivisionError[39m: division by zero
```

Um dies abzufangen können wir eine `if-else`-Verzweigung nutzen und dem Ergebnis den Wert `None` (im Sinne von ‚nicht definiert') zuweisen.

```
zaehler = 5
nenner = 0
if nenner: # false bei 0, wahr für != 0
    ergebnis = zaehler / nenner
else:
    print("Teilung durch 0")
    ergebnis = None
print(f"Das Ergebnis von {zaehler}/{nenner} = {ergebnis}")
```

```
Teilung durch 0
Das Ergebnis von 5/0 = None
```

Da Python dynamisch typisiert ist, kann man nicht ohne Weiteres garantieren, dass `zaehler` und `nenner` wirklich vom Datentyp `int` sind. Für eine verschachtelte Verzweigung kann man deshalb zunächst die Datentypen prüfen und erst innerhalb dieses gültigen Falls die Division durch 0 behandeln.

```
zaehler = 'keine_zahl'
nenner = 1
if isinstance(zaehler, int) and isinstance(nenner, int):
    if nenner != 0:
        ergebnis = zaehler / nenner
    else:
        print("Teilung durch 0")
        ergebnis = None
else:
    print("Zähler oder Nenner haben nicht den Datentyp `int`.")
    ergebnis = None

print(f"Das Ergebnis von {zaehler}/{nenner} = {ergebnis}")
```

```
Zähler oder Nenner haben nicht den Datentyp `int`.
Das Ergebnis von keine_zahl/1 = None
```

Auch dieses Beispiel lässt sich Zweig für Zweig lesen: Zuerst wird auf negative Werte geprüft. Nur wenn das nicht zutrifft, wird im `else`-Zweig weiter unterschieden, ob die Zahl positiv oder genau 0 ist.

Alternatives Verzweigen

Gibt es wie im letzten Beispiel mehrere verschachtelte Alternativen, so wird der Code sehr schnell unübersichtlich und man braucht eine kompaktere Schreibweise. Hierfür gibt es in Python die `elif`-Anweisung, welche eine Kurzform von `else if` ist und keines weiteren Einrückens bedarf. So kann man den vorherigen Code leserlicher schreiben:

```
zaehler = 'keine_zahl'
nenner = 0
ergebnis = None
if not isinstance(nenner, int):
    print(f"Nenner nicht vom Datentyp `int`, sondern vom Typ {type(nenner)}")
elif not isinstance(zaehler, int):
    print(f"Zaehler nicht vom Datentyp `int`, sondern vom Typ
{type(zaehler)}")
elif not nenner:
    print("Teilung durch 0")
else:
    ergebnis = zaehler / nenner

print(f"Das Ergebnis von {zaehler}/{nenner} = {ergebnis}")
```

```
Zaehler nicht vom Datentyp `int`, sondern vom Typ <class 'str'>
Das Ergebnis von keine_zahl/0 = None
```

Hierbei haben wir:

- als erstes die Bedingung `type(zaehler) == int` ersetzt durch die Standardfunktion `isinstance(zaehler, int)`, welche ähnlich ist und in Python meist besser lesbar.
- wir haben alle Fehlerfälle zuerst bearbeitet, um sicher zu stellen, dass sie vor Ausführung der Berechnung abgefragt werden
- so prüfen wir erst ob der `zaehler` nicht vom Datentyp `int` ist und geben in diesem Fall eine Fehlermeldung aus
- Ist das nicht der Fall, so könnte `nenner` nicht vom Datentyp `int` sein und würden in diesem Fall eine andere Fehlermeldung ausgeben
- dann muss der `nenner` vom Datentyp `int` sein und wir prüfen, ob er 0 ist, mit einer entsprechenden Fehlermeldung
- da in allen drei Fällen das `ergebnis` den Wert `None` hat, haben wir diese Zuweisung vor den `if`-Block gesetzt
- der letzte `else` Block wird nur erreicht, wenn all dies nicht zutrifft, also `zaehler` und `nenner` vom Datentypen `int` und `nenner` nicht 0 ist. Nur in diesem Fall wird `ergebnis` mit dem Wert der Teilung belegt

> **Hinweis**
>
> Hierbei ist zu beachten, dass wir absichtlich die Fehlermeldung „Teilung durch 0“ nach hinten bewegt haben. Es wäre auch möglich, mit der Bedingung `if not nenner` in der originalen Reihenfolge zu starten. Hier gäbe es allerdings die Gefahr, dass aufgrund der Wahrheitswerte anderer Datentypen auch z.B. eine leere Liste diese Bedingung erfüllen würde. Wir bekommen in diesem Fall eine falsche Fehlermeldung angezeigt („Teilung durch 0“ statt „Nenner vom Datentyp `int`“), wie folgendes Beispiel zeigt. Das ist eine der typischen Gefahren von dynamisch typisierten Sprachen. Grundsätzlich sind deshalb Datentyp-Prüfungen immer zuerst zu machen.

```
zaehler = 10
nenner = []
ergebnis = None
if not nenner:
    print("Teilung durch 0")
elif not isinstance(nenner, int):
    print(f"Nenner nicht vom Datentyp `int`, sondern vom Typ {type(nenner)}")
elif not isinstance(zaehler, int):
    print(f"Zaehler nicht vom Datentyp `int`, sondern vom Typ
{type(zaehler)}")
else:
    ergebnis = zaehler / nenner

print(f"Das Ergebnis von {zaehler}/{nenner} = {ergebnis}")
```

```
Teilung durch 0
Das Ergebnis von 10/[] = None
```

Mit `elif` bleibt dieselbe Logik erhalten, aber die Alternativen stehen auf einer Ebene. Das macht besonders bei KI-generiertem Code schneller sichtbar, welche Fälle nacheinander geprüft werden.

Wenn KI-Tools Verzweigungen generieren, ist das Ziel nicht, den Code zu schreiben, sondern ihn zu verstehen und zu überprüfen. Bei diesem Beispiel können wir konkrete Eingaben durchgehen und so jeden Zweig sichtbar machen.

Beim Lesen dieses KI-generierten Codes: (1) Bedingungen — `> 100` und `> 50`; (2) drei Zweige — „hoch“, „mittel“, „niedrig“; (3) Eingaben: `wert=150` ⟶ „hoch“, `wert=75` ⟶ „mittel“, `wert=30` ⟶ „niedrig“; (4) fehlende Fälle — `wert=None`? Negative Werte? `wert=50` exakt (nicht `> 50`, sondern `== 50`)?

Aus dem Beispiel können wir nun eine allgemeine Lesestrategie ableiten. Sie hilft dabei, Verzweigungen systematisch zu lesen, statt dem ersten Eindruck des Codes zu vertrauen.

Die Funktion `classify_stress()` klassifiziert strukturelle Spannungen in MPa. Sie wurde von einem KI-Tool generiert. Bevor wir sie in einem Ingenieurbau-Kontext verwenden, müssen wir jeden Zweig verstehen und testen.

```
def classify_stress(stress_MPa: float) -> str:
    if stress_MPa < 0:
        return "Ungültig: negativer Spannungswert"
    elif stress_MPa < 150:
        return "OK"
    elif stress_MPa < 250:
        return "Warnung"
    else:
        return "Kritisch"

print(classify_stress(-1))
print(classify_stress(100))
print(classify_stress(200))
print(classify_stress(300))
```

```
Ungültig: negativer Spannungswert
OK
Warnung
Kritisch
```

Eine einfache Tabelle hilft beim Nachverfolgen von Zweigen. Für jede Beispiel-Eingabe wird notiert, welche Bedingung zuerst zutrifft und welche Rückgabe daraus folgt. So erkennt man auch leichter fehlende Grenzfälle.

Das Minimum für die Verifikation von KI-generiertem Branching-Code ist ein Test pro Zweig. Grenzwerte (exakt 150, exakt 250) sind besonders wichtig, da < und <= häufig verwechselt werden.

```
assert classify_stress(-1)   == "Ungültig: negativer Spannungswert"
assert classify_stress(100)  == "OK"
assert classify_stress(200)  == "Warnung"
assert classify_stress(300)  == "Kritisch"
assert classify_stress(0)    == "OK"
assert classify_stress(150)  == "Warnung"
assert classify_stress(250)  == "Kritisch"
print("Alle Tests bestanden.")
```

```
Alle Tests bestanden.
```

Lesson Learned

Notizen helfen dabei, Verzweigungen aktiv zu verarbeiten. Sinnvoll sind keine kompletten Abschriften der Folien, sondern Schlüsselbegriffe, eigene Beispiele und offene Fragen, die später nachbearbeitet werden.

Nacharbeit

Gerade bei Verzweigungen lohnt sich strukturierte Nacharbeit nach dem SQ3R-Prinzip: erst den Überblick gewinnen, dann Fragen notieren, die Notizen erneut lesen, Inhalte in eigenen Worten wiedergeben und schließlich die wichtigsten Punkte wiederholen.

Quiz

Bibliographie